

Arduino samenvatting: schakelen, programmeren en toestandsdiagrammen

Elk Arduino-project bestaat uit drie stappen: **aansluiten** (de schakeling), **programmeren** (de code) en **plannen** (het toestandsdiagram). Deze samenvatting loopt ze in die volgorde door, steeds met een uitgewerkt voorbeeld dat je stap voor stap kunt nadoen.

Tip: lees een voorbeeld niet alleen, maar typ de code zelf over en verander daarna één ding. Pas als je ziet wat er verandert, snap je het echt.

Deel	Waar gaat het over?	Je kunt daarna...
1. Schakelingen	LED, knop, potmeter, lichtsensor, PWM	elk onderdeel goed aansluiten
2. Code	setup, loop, lezen, schrijven, if, millis	een programma lezen en zelf schrijven
3. Toestandsdiagrammen	toestanden, overgangen, condities	een beschrijving omzetten in een diagram
4. Van diagram naar code	vast stappenplan	een diagram omzetten in werkende code

Deel 1: Schakelingen – hoe sluit ik iets aan?

Stroom loopt alleen in een rondje: van de Arduino, door je onderdeel, terug naar **GND** (min). Elke schakeling hieronder is zo'n rondje. Klopt het rondje niet, dan werkt het niet, hoe goed je code ook is.

Eerst: welk soort onderdeel heb ik?

Stel jezelf twee vragen. Komt er informatie **binnen** (sensor, input) of verander je iets (actuator, output)? En is het signaal **digitaal** (alleen LOW/HIGH) of **analoog** (een waarde tussen bepaalde getallen)? Het antwoord bepaalt welke pin en welke code je gebruikt.

Onderdeel	Sensor of actuator	Digitaal of analoog	Welke pin?	Code
LED aan/uit	actuator (output)	digitaal	elke digitale pin, bv. 13	<code>digitalWrite(pin, HIGH)</code>

Onderdeel	Sensor of actuator	Digitaal of analoog	Welke pin?	Code
LED dimmen	actuator (output)	PWM ("nep-analoog")	alleen digitale pinnen 3, 5, 6, 9, 10, 11	<code>analogWrite(pin, 0..255)</code>
Knop	sensor (input)	digitaal	elke digitale pin, bv. 2	<code>digitalRead(pin)</code> → LOW/HIGH
Potmeter (draaiknop)	sensor (input)	analoog	A0 t/m A5	<code>analogRead(pin)</code> → 0..1023

Het breadboard

- De 5 gaatjes in één rijtje (a-e of f-j) zijn met elkaar verbonden. De middengleuf onderbreekt die verbinding.
- De lange stroken aan de rand (+ en -) zijn over de hele lengte verbonden. Daar zet je 5V en GND op, zodat elk onderdeel er makkelijk bij kan.
- Twee pootjes van hetzelfde onderdeel mogen nooit in hetzelfde verbonden rijtje staan: dan maak je kortsluiting om het onderdeel heen.

Algemeen stappenplan voor elke schakeling

1. Verbind GND van de Arduino met de - strook.
2. Zet het onderdeel op het breadboard, liefst over de middengleuf, zodat elk pootje een eigen rijtje heeft.
3. Trek een draad van de juiste Arduino-pin (zie de tabel) naar het rijtje van het juiste pootje.
4. Maak het rondje af naar GND (en eventueel 5V), met de weerstand op de juiste plek. Rood weerstandje voor LED en oranje voor een knop.
5. Controleer met je vinger: kun je vanaf de pin, door het onderdeel, bij GND komen? Dan is het rondje dicht.

Voorbeeld 1: een LED (digitale output)

1. LED op het breadboard. Let op: een LED heeft een richting. Het **lange pootje (+)** wijst naar de Arduino-pin, het **korte pootje (-)** naar GND.
2. Draad van pin 13 naar het rijtje van het lange pootje.
3. Weerstand rood van het rijtje van het korte pootje naar de - strook.
4. Twee LEDs? Doe precies hetzelfde voor de tweede LED op een andere pin (bv. 12). Elke LED krijgt een eigen weerstand.

Voorbeeld 2: een knop (digitale input)

Een knop maakt alleen contact als je hem indrukt. Losgelaten hangt de pin "in de lucht" en leest hij willekeurig LOW of HIGH. Daarom moet de pin altijd naar een vaste waarde getrokken worden. Daar zijn twee manieren voor:

	Mogelijkheid A: pull-down weerstand	Mogelijkheid B: interne pull-up
Aansluiten	één kant knop naar 5V; andere kant naar pin 2 én via een oranje weerstand naar GND	één kant knop naar pin 2; andere kant naar GND; geen weerstand
In <code>setup()</code>	<code>pinMode(2, INPUT);</code>	<code>pinMode(2, INPUT_PULLUP);</code>
Losgelaten leest	LOW	HIGH
Ingedrukt leest	HIGH	LOW (omgekeerd!)

Mogelijkheid B is minder draadjes, maar onthoud dat ingedrukt dan LOW is. In je code check je dus `if (digitalRead(2) == LOW)`.

Voorbeeld 3: een potmeter (analoge input)

1. Potmeter op het breadboard; hij heeft drie pootjes.
2. Buitenste pootje links naar de + strook (5V), buitenste pootje rechts naar de – strook (GND).
3. **Middelste** pootje naar A0.
4. Draai je aan de knop, dan leest `analogRead(A0)` een getal tussen 0 en 1023. Loopt het de verkeerde kant op? Wissel dan de twee buitenste pootjes.

Voorbeeld 4: LEDs dimmen met PWM

Aansluiten gaat precies als bij Voorbeeld 1. Het enige verschil: kies een pin uit deze lijst 3, 5, 6, 9, 10 of 11. Alleen die pinnen kunnen PWM. De Arduino zet de pin dan heel snel aan en uit. Hoe groter het deel van de tijd dat hij aan staat (de **duty cycle**), hoe feller de LED lijkt.

<code>analogWrite(pin, ...)</code>	Duty cycle	LED
0	0%	uit
64	25%	zwak

<code>analogWrite(pin, ...)</code>	Duty cycle	LED
128	50%	half
255	100%	vol aan

Voorbeeld 5: combineren (knop + LED)

Sluit elk onderdeel **los** aan, precies zoals hierboven: de knop op pin 2 (Voorbeeld 2), de LED op pin 13 (Voorbeeld 1). Verbind de knop dus niet rechtstreeks met de LED. Wat er gebeurt als je drukt, bepaalt het programma. Zo kun je later de werking veranderen zonder ook maar één draadje te verplaatsen.

Deel 2: Code – hoe programmeer ik het?

Elk Arduino-programma heeft dezelfde drie delen: variabelen bovenaan, `setup()` die één keer draait, en `loop()` die eindeloos herhaalt. Als je dat vaste skelet kent, hoef je per opdracht alleen de inhoud te bedenken.

Het vaste skelet

```
// 1. Variabelen: welke pin is wat?
const int pinLed = 13;

// 2. setup(): 1x bij het opstarten
void setup() {
  pinMode(pinLed, OUTPUT); // stel elke pin in
}

// 3. loop(): steeds opnieuw, duizenden keren per seconde
void loop() {
  // a. sensoren lezen
  // b. beslissen (if)
  // c. actuatoren zetten
}
```

Denk bij `loop()` altijd aan die drie stappen: **lezen** → **beslissen** → **zetten**. Bijna elk programma in deze samenvatting volgt dat patroon.

Variabelen en types

In JavaScript schreef je overal `var`. In Arduino (C++) moet je zeggen wat voor soort waarde erin zit.

Type	Wat zit erin	Voorbeeld
<code>int</code>	geheel getal	<code>int waardePot = 512;</code>
<code>float</code>	kommagetal (met een punt!)	<code>float temperatuur = 21.5;</code>
<code>bool</code>	<code>true</code> of <code>false</code>	<code>bool voetgangerWacht = false;</code>
<code>String</code>	tekst	<code>String naam = "Arduino";</code>

Geef pinnen altijd een naam (`pinLed` in plaats van `13`). Moet de LED naar een andere pin, dan pas je alleen die ene regel aan.

De vaste functies op een rij

Functie	Waar?	Wat doet het	Voorbeeld
<code>pinMode(pin, modus)</code>	setup	pin instellen als <code>OUTPUT</code> , <code>INPUT</code> of <code>INPUT_PULLUP</code>	<code>pinMode(pinLed, OUTPUT);</code>
<code>digitalWrite(pin, waarde)</code>	loop	pin aan (<code>HIGH</code>) of uit (<code>LOW</code>)	<code>digitalWrite(pinLed, HIGH);</code>
<code>digitalRead(pin)</code>	loop	geeft <code>HIGH</code> of <code>LOW</code> terug	<code>int knop = digitalRead(pinKnop);</code>
<code>analogRead(pin)</code>	loop	geeft 0..1023 terug (alleen A0-A5)	<code>int pot = analogRead(A0);</code>
<code>analogWrite(pin, waarde)</code>	loop	PWM 0..255 (alleen pinnen met ~)	<code>analogWrite(pinLed, 128);</code>
<code>delay(ms)</code>	loop	wacht en doet NIETS anders	<code>delay(1000);</code>
<code>millis()</code>	loop	aantal ms sinds het opstarten	<code>if (millis() - startTijd >= 1000)</code>
<code>Serial.begin(9600)</code>	setup	Serial Monitor aanzetten	<code>Serial.begin(9600);</code>
<code>Serial.println(x)</code>	loop	waarde laten zien in de Serial Monitor	<code>Serial.println(pot);</code>

Let op het verschil: `digitalRead` en `analogRead` **geven iets terug**, dus die zet je achter een `=`. `digitalWrite` en `analogWrite` **doen iets**, die staan op zichzelf.

Uitgewerkt voorbeeld 1: een LED laten knipperen

Opdracht: de LED op pin 13 gaat een halve seconde aan, een halve seconde uit, enzovoort.

1. Welke onderdelen? Alleen een LED: een actuator, digitaal. Pin 13 dus **OUTPUT**.
2. Wat moet er in **loop()** gebeuren? Aan, wachten, uit, wachten. Daarna begint **loop()** vanzelf opnieuw.

```
const int pinLed = 13;

void setup() {
  pinMode(pinLed, OUTPUT);
}

void loop() {
  digitalWrite(pinLed, HIGH); // aan
  delay(500);                 // 500 ms wachten
  digitalWrite(pinLed, LOW);  // uit
  delay(500);
}
```

Zelf aanpassen: laat hem 2 seconden aan en 0,2 seconde uit staan. Welke twee getallen verander je?

Uitgewerkt voorbeeld 2: knop bestuurt LED

Opdracht: zolang de knop (pin 2, met pull-down) is ingedrukt, brandt de LED (pin 13).

1. Onderdelen: knop = digitale input, LED = digitale output.
2. Lezen: **digitalRead(pinKnop)** geeft **HIGH** als hij is ingedrukt (pull-down, zie Deel 1).
3. Beslissen: **if** ingedrukt, **else** niet ingedrukt.
4. Zetten: LED aan of uit.

```

const int pinKnop = 2;
const int pinLed = 13;

void setup() {
  pinMode(pinKnop, INPUT);
  pinMode(pinLed, OUTPUT);
}

void loop() {
  int waardeKnop = digitalRead(pinKnop);  // lezen

  if (waardeKnop == HIGH) {               // beslissen
    digitalWrite(pinLed, HIGH);           // zetten
  } else {
    digitalWrite(pinLed, LOW);
  }
}

```

Gebruik je INPUT_PULLUP? Dan verander je twee dingen: `pinMode(pinKnop, INPUT_PULLUP);` en `if (waardeKnop == LOW)`.

Uitgewerkt voorbeeld 3: potmeter dimt een LED

Opdracht: met de potmeter (A0) regel je hoe fel de LED (pin 10) brandt.

1. Onderdelen: potmeter = analoge input (0..1023), LED = PWM-output (0..255). Pin 10 heeft een ~, dus dat kan.
2. Probleem: de input gaat tot 1023, de output maar tot 255. $1023 / 4 \approx 255$, dus je deelt door 4.
3. Met `Serial.println` kijk je mee welke waarde de potmeter geeft.

```

const int pinPot = A0;
const int pinLed = 10;

void setup() {
  pinMode(pinLed, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int waardePot = analogRead(pinPot);  // 0..1023
  Serial.println(waardePot);
  analogWrite(pinLed, waardePot / 4);  // 0..255
}

```

Open de Serial Monitor (vergrootglas rechtsboven in de Arduino IDE) en draai aan de knop. Je ziet de getallen veranderen.

Uitgewerkt voorbeeld 4: van delay naar millis

`delay()` lijkt makkelijk, maar tijdens een delay doet de Arduino **helemaal niets**. Druk je dan op een knop, dan wordt die niet gelezen. Zodra je tijd én input combineert, gebruik je daarom `millis()`.

Vergelijk het met een stopwatch die altijd doorloopt. Je wacht niet, maar je onthoudt hoe laat het was toen je begon (`startTijd`). Elke keer dat `loop()` langskomt, kijk je: is er al genoeg tijd voorbij? Dat is `millis() - startTijd`.

Stappenplan om een delay om te zetten:

1. Maak bovenaan een variabele `unsigned long startTijd = 0;`.
2. Vervang `delay(x)` door `if (millis() - startTijd >= x) { ... }`.
3. Zet binnen die `if` als eerste `startTijd = millis();` (de stopwatch opnieuw starten).
4. Zet daarna binnen de `if` wat er ná de delay moest gebeuren.

Voorbeeld 1 (knipperen) wordt dan:

```
const int pinLed = 13;
unsigned long startTijd = 0;
bool ledAan = false;

void setup() {
  pinMode(pinLed, OUTPUT);
}

void loop() {
  if (millis() - startTijd >= 500) { // 500 ms voorbij?
    startTijd = millis();           // stopwatch opnieuw
    ledAan = !ledAan;              // ! draait om: true wordt false en
    andersom                       // andersom

    if (ledAan) {
      digitalWrite(pinLed, HIGH);
    } else {
      digitalWrite(pinLed, LOW);
    }
  }
  // hier kun je nu wél een knop lezen, de loop blijft draaien
}
```

Loop het in je hoofd na. Na 0 ms: nog niets. Na 100, 200, 300 ms: de `if` is nog niet waar, `loop()` draait gewoon door. Na 500 ms: de `if` is waar, de LED gaat om en `startTijd` wordt 500. Na 1000 ms: weer 500 voorbij, de LED gaat weer om.

Uitgewerkt voorbeeld 5: een knopdruk één keer tellen

`loop()` draait duizenden keren per seconde. Houd je een knop een halve seconde vast, dan leest de Arduino honderden keren `HIGH`. Wil je dat één druk maar één keer telt (aan/uit-schakelaar, teller, start/stop), dan moet je het **moment van indrukken** vinden: nu `HIGH`, de vorige keer nog `LOW`.

```
const int pinKnop = 2;
int vorigeKnop = LOW;
int teller = 0;

void setup() {
  pinMode(pinKnop, INPUT);
  Serial.begin(9600);
}

void loop() {
  int waardeKnop = digitalRead(pinKnop);
  bool knopIngedrukt = (waardeKnop == HIGH && vorigeKnop == LOW);
  vorigeKnop = waardeKnop;           // onthouden voor de volgende keer

  if (knopIngedrukt) {
    teller = teller + 1;
    Serial.println(teller);
  }
}
```

`&&` betekent "en": beide moeten waar zijn. Dit trucje heb je straks nodig bij toestandsdiagrammen met een knop. Telt een echte knop soms dubbel? Dat komt doordat de metalen contactjes even "stuiteren". Een korte `delay(20);` aan het eind van `loop()` lost dat meestal op.

Deel 3: Toestandsdiagrammen – hoe maak ik er een?

Een toestandsdiagram is een tekening van alles wat een apparaat kan "zijn" en wanneer het overspringt. Je maakt het **vóór** je gaat programmeren: eerst denken, dan typen. Het werkt goed voor eenvoudige apparaten zoals een stoplicht, stopwatch, tandenborstel of kluis.

De drie bouwstenen

Bouwsteen	Tekening	Betekenis	Voorbeeld
Toestand	rondje met naam	hoe het apparaat nu staat	ROOD , TIJD_LOOPT
Overgang	pijl van rondje naar rondje	het apparaat springt naar een andere toestand	ROOD → GROEN
Conditie	tekst bij de pijl	wanneer de overgang gebeurt	"24 sec voorbij", "knop ingedrukt"

En één startpijl (komt uit het niets) naar de toestand waarin het apparaat begint.

Stappenplan: van beschrijving naar diagram

1. **Toestanden zoeken.** Lees de beschrijving en onderstreep wat het apparaat *is* of *doet* ("staat op rood", "de tijd loopt"). Elk daarvan is een kandidaat-toestand.
2. **Condities zoeken.** Omcirkel alles waardoor het verandert: "na 10 seconde", "als je op de knop drukt", "als het donker wordt".
3. **Rondjes tekenen.** Eén rondje per toestand, met een korte naam.
4. **Startpijl.** Welke toestand is het eerst, als je het apparaat aanzet?
5. **Pijlen tekenen.** Ga elke toestand langs en vraag: wat kan hier gebeuren, en waar ga ik dan heen? Zet de conditie bij de pijl.
6. **Controleren.** Speel het apparaat na met je vinger op het diagram. Doet het precies wat de beschrijving zegt? Heeft elke toestand een pijl naar buiten (of hoort hij juist te blijven staan)?

Vier regels die vaak fout gaan

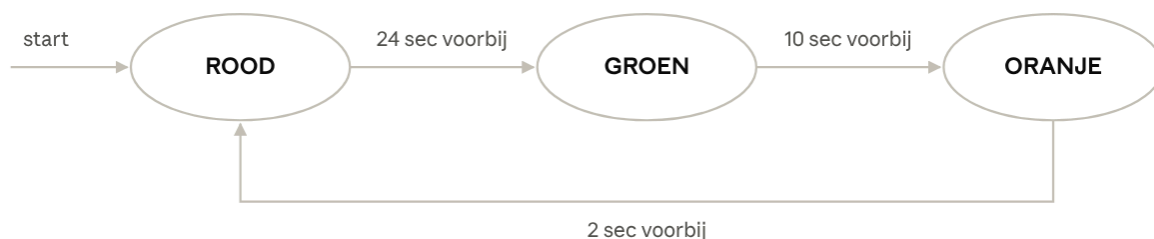
- **Een toestand bevat alle informatie** die nodig is om het apparaat later weer in dezelfde stand te zetten.
- **Zelfde uiterlijk, toch twee toestanden.** Een Duits stoplicht staat twee keer op oranje: voor groen en voor rood. Daarna gaat het een andere kant op, dus dat zijn twee toestanden: `ORANJE_NA_ROOD` en `ORANJE_NA_GROEN`.
- **Niet elke combinatie hoeft.** Bij een auto- plus voetgangersstoplicht zijn 9 combinaties denkbaar, maar groen + groen mag nooit (botsing). Je tekent alleen de combinaties die echt voorkomen.
- **Tel niet met toestanden.** Een stopwatch met een toestand per seconde wordt oneindig groot. Tellen doe je in de code met een variabele, niet met rondjes.

Uitgewerkt voorbeeld 1: stoplicht

Beschrijving: het stoplicht staat achtereenvolgens 24 seconde op rood, 10 seconde op groen en 2 seconde op oranje. Als je het aanzet, begint het met rood.

1. Toestanden: rood, groen, oranje → **ROOD** , **GROEN** , **ORANJE** .
2. Conditie: "24 sec voorbij", "10 sec voorbij", "2 sec voorbij".
3. Start: "begint met rood" → startpijl naar **ROOD** .
4. Pijlen: vanuit **ROOD** na 24 sec naar **GROEN** , vanuit **GROEN** na 10 sec naar **ORANJE** , vanuit **ORANJE** na 2 sec terug naar **ROOD** .

Het stoplicht loopt steeds rond: **ROOD, GROEN, ORANJE, ROOD**



toestandsdiagram stoplicht · 3 toestanden

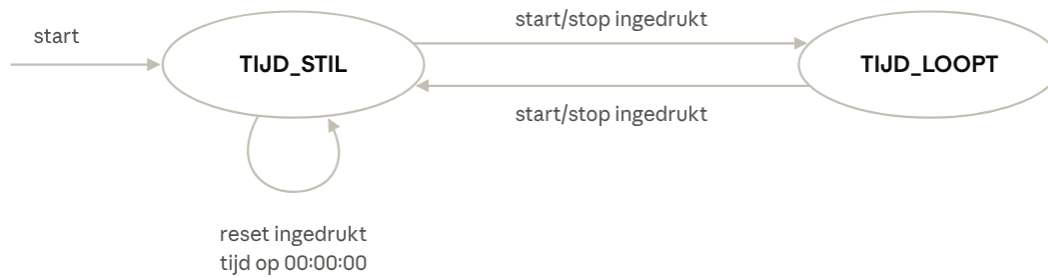
Controle met je vinger: start bij **ROOD** , wacht 24 s, **GROEN** , wacht 10 s, **ORANJE** , wacht 2 s, weer **ROOD** . Klopt met de beschrijving.

Uitgewerkt voorbeeld 2: stopwatch

Beschrijving: de stopwatch heeft een knop start/stop en een knop reset. Na het aanzetten staat hij stil op 00:00:00. Start/stop laat de tijd lopen, nog een keer drukken zet hem stil, nog een keer laat hem verder lopen. Reset zet de tijd op 00:00:00, maar alleen als hij stilstaat.

1. Toestanden: de tijd staat stil, of de tijd loopt → **TIJD_STIL** , **TIJD_LOOPT** . Niet een toestand per seconde (regel 4): de tijd zelf houdt je bij in een variabele.
2. Conditie: "start/stop ingedrukt", "reset ingedrukt".
3. Start: "staat stil" → startpijl naar **TIJD_STIL** .
4. Pijlen: start/stop gaat heen en weer. Reset in **TIJD_STIL** geeft een pijl naar zichzelf terug, met als actie "tijd op 00:00:00". In **TIJD_LOOPT** doet reset niets, dus daar komt geen reset-pijl.

Start/stop wisselt heen en weer; reset werkt alleen in TIJD_STIL



toestandsdiagram stopwatch · 2 toestanden

Geen pijl betekent: er gebeurt niets. Druk je reset terwijl de tijd loopt, dan blijf je gewoon in `TIJD_LOOPT`.

Deel 4: Van toestandsdiagram naar code

Een toestandsdiagram zet je altijd op dezelfde manier om in code. Elk onderdeel van de tekening heeft een vaste plek in je programma. Leer deze vertaaltabel en je kunt elk diagram programmeren.

In het diagram	In de code	Waar?
Toestand (rondje)	<code>const int ROOD = 1;</code>	bovenaan
Startpijl	<code>int toestand = ROOD;</code>	bovenaan
Conditie met tijd	<code>millis() - toestandStartTijd >= 24000</code>	loop, deel B
Conditie met knop	<code>knopIngedrukt</code> (zie Deel 2, voorbeeld 6)	loop, deel A en B
Overgang (pijl)	<code>toestand = GROEN;</code> en <code>toestandStartTijd = millis();</code>	loop, deel B
Actie bij een pijl ("buzz kort")	een opdracht binnen die overgang	loop, deel B
Wat je ziet in een toestand	<code>digitalWrite(...)</code> per toestand	loop, deel C

Stappenplan

- 1. Toestanden als constanten.** Geef elke toestand een nummer: `const int ROOD = 1;`.
Dan kun je in je code `ROOD` schrijven in plaats van een los getal.
- 2. Begintoestand.** De startpijl wordt `int toestand = ROOD;`. Gebruik je tijd? Maak ook `unsigned long toestandStartTijd = 0;`.
- 3. setup().** Zet alle pinnen met `pinMode`.
- 4. loop() deel A: sensoren lezen.** Lees alle knoppen en sensoren bovenaan, één keer.
- 5. loop() deel B: toestand bepalen.** Schrijf per toestand een `if (toestand == ...)`. Zet daarbinnen per pijl die uit dat rondje vertrekt een `if (conditie)`. Is de conditie waar, dan: nieuwe toestand invullen en `toestandStartTijd = millis();`.
- 6. loop() deel C: actuatoren zetten.** Schrijf per toestand welke LEDs (of motor, buzzer) aan en uit moeten.

Waarom `else if` in deel B? Zonder `else` kan de Arduino in één rondje van `loop()` twee pijlen achter elkaar nemen. Bij de stopwatch zou één druk op start/stop je van `TIJD_STIL` naar `TIJD_LOOPT` brengen, en in hetzelfde rondje meteen weer terug. Met `else if` wordt per rondje maar één toestand bekeken.

Uitgewerkt voorbeeld 1: stoplicht (alleen tijd)

We programmeren het diagram uit Deel 3, Uitgewerkt voorbeeld 1. Rood zit op pin 12, oranje op 10, groen op 8.

Stap 1 en 2: constanten en begintoestand. Drie rondjes geeft drie constanten; de startpijl wijst naar `ROOD`.

Stap 3: setup. Drie LEDs, dus drie keer `OUTPUT`.

Stap 4: sensoren lezen. Dit stoplicht heeft geen knop, dus deel A blijft leeg.

Stap 5: toestand bepalen. Elk rondje heeft precies één pijl naar buiten, met tijd als conditie.

Stap 6: actuatoren zetten. In elke toestand brandt precies één LED.

```
// pinnen
const int pinRood = 12;
const int pinOranje = 10;
const int pinGroen = 8;

// stap 1: toestanden
const int ROOD = 1;
const int GROEN = 2;
const int ORANJE = 3;
```

```
// stap 2: begintoestand (de startpijl)
int toestand = ROOD;
unsigned long toestandStartTijd = 0;

// stap 3
void setup() {
  pinMode(pinRood, OUTPUT);
  pinMode(pinOranje, OUTPUT);
  pinMode(pinGroen, OUTPUT);
}

void loop() {
  // stap 4 (A): sensoren lezen -> geen sensoren

  // stap 5 (B): toestand bepalen
  if (toestand == ROOD) {
    if (millis() - toestandStartTijd >= 24000) { // pijl: 24 sec voorbij
      toestand = GROEN;
      toestandStartTijd = millis();
    }
  } else if (toestand == GROEN) {
    if (millis() - toestandStartTijd >= 10000) { // pijl: 10 sec voorbij
      toestand = ORANJE;
      toestandStartTijd = millis();
    }
  } else if (toestand == ORANJE) {
    if (millis() - toestandStartTijd >= 2000) { // pijl: 2 sec voorbij
      toestand = ROOD;
      toestandStartTijd = millis();
    }
  }

  // stap 6 (C): actuatoren zetten
  if (toestand == ROOD) {
    digitalWrite(pinRood, HIGH);
    digitalWrite(pinOranje, LOW);
    digitalWrite(pinGroen, LOW);
  } else if (toestand == GROEN) {
    digitalWrite(pinRood, LOW);
    digitalWrite(pinOranje, LOW);
    digitalWrite(pinGroen, HIGH);
  } else if (toestand == ORANJE) {
    digitalWrite(pinRood, LOW);
  }
}
```

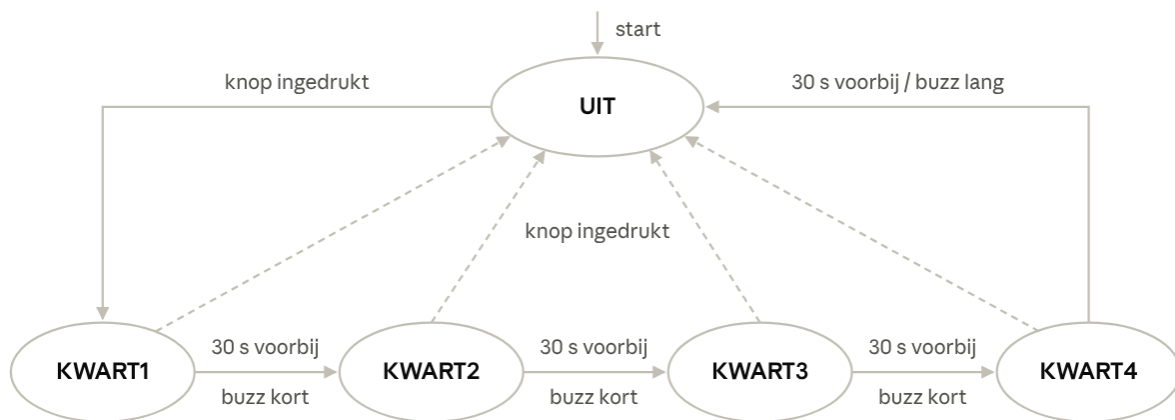
```
digitalWrite(pinOranje, HIGH);
digitalWrite(pinGroen, LOW);
}
}
```

Leg het diagram naast de code: elk rondje is een `if (toestand == ...)`, elke pijl een `if` daarbinnen, elk getal op een pijl staat terug in de code. Geen enkele `delay()`, dus je kunt later makkelijk een voetgangersknop toevoegen.

Uitgewerkt voorbeeld 2: tandenborstel (knop én tijd)

Beschrijving: de tandenborstel begint uit. Druk je op de knop, dan gaat hij aan en loopt de tijd. Na 30, 60 en 90 seconde buzzt hij kort, na 120 seconde lang en dan gaat hij uit. Druk je tijdens het poetsen op de knop, dan gaat hij meteen uit.

Tandenborstel: vier kwarten van 30 seconden, de knop zet hem altijd uit



toestandsdiagram tandenborstel · 5 toestanden, gestippeld = knop

Het tellen gebeurt hier wél met toestanden: dat mag, want het zijn er maar vier (beperkt aantal). De knop zit op pin 2 (pull-down), een LED op pin 13 speelt de motor, een buzzer zit op pin 8.

Stap 1 en 2. Vijf rondjes, dus vijf constanten. Start in `UIT`.

Stap 4: sensoren lezen. We willen één druk één keer tellen, dus we gebruiken de truc van Deel 2, voorbeeld 6 (`knopIngedrukt`).

Stap 5: toestand bepalen. Kijk per rondje welke pijlen vertrekken. Uit `UIT`: één pijl (knop). Uit elk kwart: twee pijlen, de knop (naar `UIT`) en de tijd (naar het volgende kwart). De actie "buzz kort" zet je binnen de tijd-overgang. Voor de buzzer gebruiken we `tone(pin, toonhoogte, duur)`. Die blokkeert niet, in tegenstelling tot `delay()`.

Stap 6: actuatoren zetten. Alleen in `UIT` staat de motor uit.

```

const int pinKnop = 2;
const int pinMotor = 13;    // LED speelt de motor
const int pinBuzzer = 8;

// stap 1: toestanden
const int UIT = 0;
const int KWART1 = 1;
const int KWART2 = 2;
const int KWART3 = 3;
const int KWART4 = 4;

// stap 2: begintoestand
int toestand = UIT;
unsigned long toestandStartTijd = 0;
int vorigeKnop = LOW;

// stap 3
void setup() {
    pinMode(pinKnop, INPUT);
    pinMode(pinMotor, OUTPUT);
    pinMode(pinBuzzer, OUTPUT);
}

void loop() {
    // stap 4 (A): sensoren lezen
    int waardeKnop = digitalRead(pinKnop);
    bool knopIngedrukt = (waardeKnop == HIGH && vorigeKnop == LOW);
    vorigeKnop = waardeKnop;

    // stap 5 (B): toestand bepalen
    if (toestand == UIT) {
        if (knopIngedrukt) {
            toestand = KWART1;
            toestandStartTijd = millis();
        }
    } else if (toestand == KWART1) {
        if (knopIngedrukt) {
            toestand = UIT;
        } else if (millis() - toestandStartTijd >= 30000) {
            tone(pinBuzzer, 1000, 200);    // buzz kort
            toestand = KWART2;
            toestandStartTijd = millis();
        }
    } else if (toestand == KWART2) {
        if (knopIngedrukt) {
            toestand = UIT;
        }
    }
}

```

```

    } else if (millis() - toestandStartTijd >= 30000) {
        tone(pinBuzzer, 1000, 200);
        toestand = KWART3;
        toestandStartTijd = millis();
    }
} else if (toestand == KWART3) {
    if (knopIngedrukt) {
        toestand = UIT;
    } else if (millis() - toestandStartTijd >= 30000) {
        tone(pinBuzzer, 1000, 200);
        toestand = KWART4;
        toestandStartTijd = millis();
    }
} else if (toestand == KWART4) {
    if (knopIngedrukt) {
        toestand = UIT;
    } else if (millis() - toestandStartTijd >= 30000) {
        tone(pinBuzzer, 1000, 1000);    // buzz lang
        toestand = UIT;
    }
}

// stap 6 (C): actuatoren zetten
if (toestand == UIT) {
    digitalWrite(pinMotor, LOW);
} else {
    digitalWrite(pinMotor, HIGH);
}
}

```

Test het zonder 2 minuten te wachten: verander tijdelijk elke 30000 in 3000 . Dan duurt een poetsbeurt 12 seconden.